

Assignment 5

Nischal Pant

2023-11-19

6. We continue to consider the use of a logistic regression model to predict the probability of default using income and balance on the Default data set. In particular, we will now compute estimates for the standard errors of the income and balance logistic regression coefficients in two different ways: (1) using the bootstrap, and (2) using the standard formula for computing the standard errors in the `glm()` function. Do not forget to set a random seed before beginning your analysis.
- (a) Using the `summary()` and `glm()` functions, determine the estimated standard errors for the coefficients associated with income and balance in a multiple logistic regression model that uses both predictors

```
library(ISLR2)

## Warning: package 'ISLR2' was built under R version 4.2.3

set.seed(221)
fit.mod <- glm(default ~ income + balance, data = Default, family =
"binomial")
summary(fit.mod)$coef[, "Std. Error"]

## (Intercept)      income      balance
## 4.347564e-01 4.985167e-06 2.273731e-04

glm.se <- summary(fit.mod)$coef[, "Std. Error"]
```

- (b) Write a function, `boot.fn()`, that takes as input the Default data set as well as an index of the observations, and that outputs the coefficient estimates for income and balance in the multiple logistic regression model.

```
boot.fn <- function(data, indices) {
  sample_data <- data[indices, ]
  model <- glm(default ~ income + balance, data = sample_data, family =
"binomial")
  return(coef(model))
}
```

- (c) Use the `boot()` function together with your `boot.fn()` function to estimate the standard errors of the logistic regression coefficients for income and balance.

```
library(boot)
set.seed(423)

boot.strp <- boot(data = Default, statistic = boot.fn, R = 1000)
boot.se <- apply(boot.strp$t, 2, sd)
```

- (d) Comment on the estimated standard errors obtained using the `glm()` function and using your bootstrap function.

```
print("Standard Errors (glm):")
## [1] "Standard Errors (glm):"
print(summary(fit.mod)$coef[, "Std. Error"])
## (Intercept)      income      balance
## 4.347564e-01 4.985167e-06 2.273731e-04
print("Standard Errors (bootstrap):")
## [1] "Standard Errors (bootstrap):"
print(boot.se)
## [1] 4.505711e-01 4.926907e-06 2.335848e-04
```

The standard error for the bootstrap function is less compared to `glm` making it much more precise.

9. We will now consider the Boston housing data set, from the `ISLR2` library.

- (a) Based on this data set, provide an estimate for the population mean of `medv`. Call this estimate $\mu^\wedge$.

```
data(Boston)
mu.h <- mean(Boston$medv)
mu.h
## [1] 22.53281
```

- (b) Provide an estimate of the standard error of $\mu^\wedge$. Interpret this result. Hint: We can compute the standard error of the sample mean by dividing the sample standard deviation by the square root of the number of observations.

```
se.mu.h <- sd(Boston$medv) / sqrt(length(Boston$medv))
se.mu.h
## [1] 0.4088611
```

Interpretation: The standard error of the sample mean provides a measure of the uncertainty or variability in our estimate of the population mean. A larger standard error suggests more variability and less precision in our estimate.

- (c) Now estimate the standard error of $\mu^\wedge$ using the bootstrap. How does this compare to your answer from (b)?

```
boot_fn <- function(data, indices) {
  sample_data <- data[indices]
  return(mean(sample_data))
}
```

```
boot_results <- boot(data = Boston$medv, statistic = boot_fn, R = 1000)

se.boot.mu.h <- sd(boot_results$t)
se.boot.mu.h

## [1] 0.4207522
```

The bootstrap result for se is higher suggests there is more variability and less precision of the mu.h.

- (d) Based on your bootstrap estimate from (c), provide a 95 % confidence interval for the mean of medv. Compare it to the results obtained using `t.test(Boston$medv)`. Hint: You can approximate a 95 % confidence interval using the formula $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$.

```
conf.int.boot <- quantile(boot_results$t, c(0.025, 0.975))
conf.int.boot

##      2.5%      97.5%
## 21.73696 23.37487

t.conf.int <- t.test(Boston$medv)$conf.int
t.conf.int

## [1] 21.72953 23.33608
## attr(,"conf.level")
## [1] 0.95
```

- (e) Based on this data set, provide an estimate, $\hat{\mu}_{med}$, for the median value of medv in the population.

```
med.h <- median(Boston$medv)
med.h

## [1] 21.2
```

- (f) We now would like to estimate the standard error of $\hat{\mu}_{med}$. Unfortunately, there is no simple formula for computing the standard error of the median. Instead, estimate the standard error of the median using the bootstrap. Comment on your findings.

```
boot_median_fn <- function(data, indices) {
  sample_data <- data[indices]
  return(median(sample_data))
}

boot.med <- boot(data = Boston$medv, statistic = boot_median_fn, R = 1000)
se.boot.med.h <- sd(boot.med$t)
se.boot.med.h

## [1] 0.361759
```

- (g) Based on this data set, provide an estimate for the tenth percentile of medv in Boston census tracts. Call this quantity $\hat{\mu}^{0.1}$. (You can use the `quantile()` function.)

```
p10.h <- quantile(Boston$medv, 0.1)
p10.h

## 10%
## 12.75
```

(h) Use the bootstrap to estimate the standard error of $\mu^{0.1}$. Comment on your findings.

```
boot.p10 <- function(data, indices) {
  sample_data <- data[indices]
  return(quantile(sample_data, 0.1))
}

boot.p10.fin <- boot(data = Boston$medv, statistic = boot.p10, R = 1000)

#
se.boot.p10.h <- sd(boot.p10.fin$t)
se.boot.p10.h

## [1] 0.4927669
```

There is a moderate level of variability since our result was 0.5003814.